

PhD Training School in Poznan
Aleksandra Rutkowska

Research Infrastructure Toolkit

version control, data storage, experiment tracking, and data sources for a quantitative finance PhD



Day 1 — Full Schedule

September 21, 2026 · 09:00–17:00 · Foundations of Machine Learning and AI in Finance

09:00–10:30	Part 0 — AI, ML & the paradigms of learning	<i>This deck</i>
10:45–12:15	Part A — AI, ML & the paradigms of learning	<i>This deck</i>
12:15–13:15	<i>Lunch break</i>	
13:15–14:45	Part B — Finance-specific challenges	<i>This deck</i>
15:00–16:15	Case Study 1 — Fraud: scale, taxonomy, sources	<i>Case Study 1 deck</i>
16:15–17:00	Exercises: dataset-landscape notebook + source-literacy briefing memo	<i>Notebook + handout</i>

Blocks 1–3 are this deck; Block 4 hands off to the Case Study 1: Financial Fraud Detection deck; Block 5 uses the companion notebook and handout.

Why set this up before main Day 1

The course teaches methods. This toolkit is what keeps your results reproducible while you use them.

1

Every notebook this week produces a result

Day 2's model comparisons, Day 3's paper replication — all depend on knowing exactly which data and code version produced which number.

2

The failure mode is invisible until it isn't

"Which settings gave Table 2?" is the question that always comes up at your defense — months after you've forgotten, without a system that remembers for you.

3

Retrofitting is much more painful than setting up first

Adding version control to six months of unstructured notebooks is a research project in itself. Five tools, configured now, prevent that entirely.

PART A

Version Control — Code & Data

Git, with research habits

<https://git-scm.com>

<https://github.com>

Workflow habits

- **Branch per experiment** — not per person. Makes it trivial to abandon a dead end without losing history.
- **Tag code versions** that correspond to submitted chapters or paper drafts — "v1-submission", not just the latest commit.
- **Commit messages describe why**, not what — the diff already shows what changed.

Pre-commit hooks

black / ruff — auto-format and lint Python before every commit, so style debates never happen in review.

nbstripout — strips notebook outputs before commit. Without it, every plot you render bloats the repo and every diff becomes unreadable.

```
Setup: pip install pre-commit, then pre-commit install  
once per repo.
```

DVC — version control for data

<https://dvc.org/>

The question DVC answers: "Exactly which version of the data produced Table 3 in my paper?" Financial data restates — splits, dividend adjustments, delayed corrections to fundamentals. Without a data snapshot, you cannot reproduce your own result from six months ago, let alone defend it under review.

How it works

Works like Git, for large files: your repo stores a small pointer file; the actual data lives in S3, GCS, or a local/network drive. Commit the pointer, and the data version travels with your code version.

Lighter alternative

`git-lfs` does something similar with less setup, but fewer features (no built-in pipelines, weaker at tracking derived datasets). Fine for a handful of moderately-sized files; DVC scales further as your data pipeline grows.

PART B

Data Storage & Preparation

Fast, typed, queryable — and reproducible by construction

Parquet & DuckDB

A file format and a query engine that make time-series work dramatically faster

Parquet, instead of CSV

- Columnar storage — reading only the columns you need is dramatically faster than parsing a whole CSV row by row.
- Preserves types — dates stay dates, no re-parsing "2024-01-15" as a string every time you load the file.
- Built-in compression — a day of tick data in Parquet is a fraction of the CSV size.

DuckDB

A serverless analytical database — no server to install or manage, just a Python import.

Query Parquet/CSV files directly with SQL, including joins across files, without loading everything into memory first.

Where it fits: the natural home for a growing collection of downloaded financial datasets you want to query, not just files you load once and discard.

Parquet & DuckDB

A file format and a query engine that make time-series work dramatically faster

Why Parquet?

- **Smaller files:** Parquet uses columnar storage and compression, often reducing file size significantly compared to CSV.
- **Faster reads:** You can load only the columns you need instead of reading the entire file.

Why DuckDB?

- **Query data directly from Parquet:** No need to load the whole dataset into memory first.
- **SQL on local files:** Run fast analytical queries using familiar SQL syntax.

```
1 import duckdb
2
3 result = duckdb.sql("""
4     SELECT country,
5           SUM(sales) AS total_sales
6     FROM 'sales.parquet'
7     GROUP BY country
8 """).df()
9
10 print(result)
```

Parquet stores data more efficiently than CSV, and DuckDB lets you analyze Parquet files directly with SQL without loading everything into memory.

Reproducible data pipelines

A pipeline is a script that runs top to bottom, not a notebook you clicked through in a particular order

The trap: a notebook run out of cell order — or one where you tweaked cell 12 after already running cell 20 — produces a result that looks fine but can't be regenerated. This is the notebook-equivalent of the look-ahead bias problem from Day 3: the output is only trustworthy if the process that made it is provably repeatable.

The fix: scripts, not click-order

Convert the notebook's logic into a .py script (or a small set of them) once it stabilizes. Running the script top to bottom is the definition of "reproducible" — a notebook re-run in a different cell order is not.

papermill

Runs a notebook end-to-end with injected parameters — e.g. the same analysis notebook, executed once per ticker or per date range, each run saved as its own output notebook. A practical middle ground between a plain notebook and a full orchestrator (Airflow, Prefect), which is usually overkill for a PhD project.

Reproducible data pipelines

A pipeline is a script that runs top to bottom, not a notebook you clicked through in a particular order

The trap: a notebook run out of cell order

1. Run the Notebook End-to-End: Restart Kernel → Run All

2. Execute Cells in Order

3. Set Random Seeds

```
import random
import numpy as np
```

4. Version Your Data

```
random.seed(42)
np.random.seed(42)
```

5. Pin Dependencies

```
pandas==2.3.2
numpy==2.3.2
scikit-learn==1.7.1
```

6. Avoid Hidden State

7. Move Business Logic into Functions or Modules

8. Treat Notebooks Like Code

```
git add analysis.ipynb
git commit -m "Add feature
engineering step"
```

PART C

Environment & Experiment Tracking

Same code, same data, same environment — and a record of every run

Environments: conda and Docker

Two layers of the same problem: "it works on my machine"

conda / mamba

Pin every package version in an `environment.yml` file, committed to the repo alongside the code.

Without this: "it works on my machine" is not a joke — it's the default outcome of two computers with different package versions running "the same" code.

`mamba` is a drop-in, much faster reimplementation of `conda` — worth using by default.

Docker

One layer further: pins the OS and system libraries too, not just Python packages.

When it's worth the setup cost: when a result is heading toward publication, and a reviewer — or you, in two years — needs to reproduce it exactly, on any machine.

When it's not: early exploratory work. `conda` alone is usually enough until a result needs to survive contact with someone else's machine.

Experiment tracking — the most under-used tool here

<https://mlflow.org/docs/latest/ml/getting-started/quickstart/>

MLflow (self-hosted, free) or **Weights & Biases** (free academic plan) log every run automatically: hyperparameters, metrics, the data version, the code version — all tied together, per experiment, with no manual spreadsheet.

Why this matters, concretely

After three months testing GARCH vs. LSTM vs. something in between — dozens of variants, each with different hyperparameters — the question "**which exact settings produced the result in Table 2?**" always comes up: in a committee meeting, in peer review, or from your own future self. Without experiment tracking, that question has no answer. With it, it's a two-second lookup.

PART D

Financial Data & Backtesting Tools

Where the data comes from, and how to avoid re-introducing Day 3's mistakes

Financial data sources: exploration vs. publication

Not every source is appropriate for every stage of the work

Source	Cost	Best for	Watch out for
WRDS (CRSP, Compustat, IBES, TAQ)	Institutional license	Publication-grade results — what top finance journals expect	Requires university subscription; steep initial learning curve
Kenneth French Data Library	Free	Standard factor data (Fama-French factors) — widely cited, drop-in ready	Limited to the specific factors and portfolios provided
FRED	Free	Macroeconomic series — rates, inflation, GDP — official and well-documented	Not built for firm- or security-level data
yfinance / free APIs	Free	Fast prototyping — exactly what our course notebooks used	Data quality and consistency issues reviewers will find; not for final results

House rule: prototype freely on free sources — that's exactly what this course's notebooks did — but budget time to migrate to WRDS-quality data before any result goes into a chapter or submission.

Backtesting frameworks

Off-the-shelf protection against Day 3's central warning

Recall Day 3: look-ahead bias is one of the easiest mistakes to make and one of the hardest to notice by eye. A dedicated backtesting framework enforces point-in-time correctness structurally — the library's design makes the bug much harder to introduce by accident, instead of relying on you catching it in a code review.

vectorbt

Vectorized backtesting built on pandas/numpy — fast enough to test thousands of parameter combinations, with built-in walk-forward tools.

backtrader

Event-driven engine that processes data bar-by-bar, closer to how a live trading system would actually see the data arrive.

zipline-reloaded

A maintained fork of Quantopian's original engine — mature, widely used in the academic literature you'll be citing.

PART E

Compute & Literature Management

The last two pieces — plan for them before you need them urgently

Compute access & literature management

Neither is urgent in week one — both take weeks to arrange if you wait until you need them

Compute

University HPC cluster: request access in your first week — approval and onboarding routinely take longer than expected, and you don't want to be blocked the day you actually need a GPU.

Cloud research credits: Google Cloud Research Credits and Azure for Students exist specifically for this — worth knowing about before anyone pays out of pocket.

Zotero

Free reference manager with solid Word, LaTeX, and Overleaf integration.

Why it matters for this course specifically: every deck this week has cited real papers — Chawla (2002), Fischer & Krauss (2018), Liu, Ting & Zhou (2008), and more. Over 4+ years of a PhD, that citation list grows into the hundreds; a reference manager is what keeps it from becoming unmanageable.

Your Day 0 checklist

Work through this before September 21 — bring questions to Day 1's live check-in



Git installed, GitHub/GitLab account created, one practice repo pushed



pre-commit installed, with black/ruff/nbstripout configured in a sample repo



DVC installed; walked through the "init + add + push" tutorial once



conda/mamba installed; created one environment.yml and rebuilt the environment from it



MLflow or Weights & Biases account created; logged one dummy run



WRDS access requested through your university (if not already active) — this one has the longest lead time



Zotero installed, connected to Word/LaTeX/Overleaf as relevant to your workflow

Key takeaways



Version control splits in two for research: Git for code, DVC for data — both are needed, neither substitutes for the other.



Experiment tracking (MLflow / W&B) is the single most under-used tool here, and the one most likely to save you at your defense.



Prototype on free data sources, exactly like this course's notebooks — but budget time to migrate to publication-grade sources before results go into a chapter.



Set this up once, now. Retrofitting it onto six months of unstructured work is far more painful than configuring five tools before you start.

Next: September 21

Day 1, Block 1 opens with a 25-minute live check-in on this toolkit — bring whatever didn't work — before moving into Foundations of Machine Learning and AI in Finance.